

Checklist

Growing Platforms

1. Platform Evolution

- ☐ Identify the platform's target users and addressable market.
- ☐ Assess **market reach**: which user groups, geographies, or business units should adopt the platform?
- ☐ Assess **platform breadth**: does the platform cover the right portions of the problem space?
- ☐ Assess **platform depth**: are key capabilities sufficiently complete, integrated, automated, and polished?
- ☐ Balance reach, breadth, and depth rather than optimizing only one.
- ☐ Avoid building the “perfect” platform for one customer before gaining broader adoption.
- ☐ Grow the platform iteratively using real user feedback.
- ☐ Prefer cohesion over simply accumulating more features.
- ☐ Make platform trade-offs and priorities visible to users.

2. Product Lifecycle

- ☐ **Explore**: test hypotheses with inexpensive experiments.
- ☐ Validate whether users actually value the platform—not merely whether the technology works.
- ☐ **Expand**: remove obstacles preventing more users from adopting the platform.
- ☐ Improve self-service capabilities.
- ☐ Reduce dependence on manual or in-person support.
- ☐ Provide high-quality documentation.
- ☐ **Extract**: once adoption is established, improve the platform's economics and benefit from economies of scale.
- ☐ Reassess whether the team, skills, or leadership need to change between lifecycle phases.

3. Avoid Growth Pitfalls

- ☐ Avoid excessive up-front development before users interact with the platform.
- ☐ Get real users involved early enough to generate useful feedback.
- ☐ Avoid adding every feature requested by prospective users.
- ☐ Check whether expanding breadth will dilute quality or cohesion.
- ☐ Avoid adding sophisticated depth that benefits only a narrow customer segment.
- ☐ Consider the cognitive load that advanced features create for everyone else.
- ☐ Develop in iterations rather than attempting to complete the entire platform at once.

4. User Experience & On-Ramp

- ☐ Evaluate the platform from a new user's perspective.
- ☐ Measure the initial learning effort required to achieve something useful.
- ☐ Minimize the initial **cliff** that can discourage new adopters.
- ☐ Build on concepts and tools that are already familiar to potential users, where practical.
- ☐ Supply sensible defaults for complicated or rarely used parameters.
- ☐ Provide templates for common use cases.
- ☐ Provide documentation and self-guided training.
- ☐ Give users useful feedback when their actions fail.
- ☐ Make simple tasks easy while keeping complex tasks possible.
- ☐ Check for a **hockey-stick** experience where simple tasks are easy but slightly more advanced needs become dramatically harder.
- ☐ Provide programmatic interfaces when graphical interfaces will not scale to advanced development needs.
- ☐ Where users must "shift gears," make transitions between approaches as painless as possible.
- ☐ Provide migration or escape-hatch mechanisms when platform constraints are reached.

5. Platform Visualization

- ☐ Clearly show what the **platform provides** versus what the **user/project must provide**.
- ☐ Visualize the onboarding process and responsibilities when several teams are involved.

- ☐ Use capability maps to communicate scope, but do not rely on them alone.
- ☐ Use lifecycle maps when capabilities need to be understood across development, build, test, deployment, and operations.
- ☐ Show dependencies between platform components.
- ☐ Show mandatory, replaceable, and extensible components.
- ☐ Clarify ownership, provisioning, and operational responsibilities.
- ☐ Show connectivity and data flows where they are important.
- ☐ Make extension points visible.
- ☐ Use visual differences only where meaningful differences actually exist.
- ☐ Design diagrams so the overall message is obvious first and details emerge afterward.
- ☐ Explain the architecture before introducing individual products or vendor services.

6. Platform Roadmap

- ☐ Maintain a clear product vision.
- ☐ Evaluate requests against both **business impact** and **roadmap fit**.
- ☐ Prioritize high-impact requests that already fit the strategy.
- ☐ Investigate high-impact requests that challenge the current strategy.
- ☐ Add lower-impact, strategy-aligned requests to the backlog according to capacity.
- ☐ Decline low-impact requests that do not align with the platform's direction.
- ☐ Explain the principles behind declined requests.
- ☐ Avoid turning the platform team into a professional services team for one customer.
- ☐ Consider extensions, plug-ins, or separate projects for needs that should not enter the platform core.
- ☐ Review requests across the whole platform before duplicating functionality.
- ☐ Approach users with hypotheses and options rather than simply asking what features they want.
- ☐ Remember that existing users are a biased sample; consider the needs of non-users too.
- ☐ Publish a credible roadmap so users understand the platform's direction and trade-offs.

7. Metrics & Adaptation

- ☐ Track adoption and usage patterns.
- ☐ Measure outcomes and customer experience—not only platform activity.
- ☐ Use telemetry to understand platform reliability and stability.
- ☐ Look for shifts in user behavior that may create new platform needs.
- ☐ Monitor relevant external technology changes.
- ☐ Evaluate trends against the platform strategy before reacting to them.
- ☐ Avoid chasing every new technology trend.
- ☐ Revisit the roadmap regularly as technology, users, and organizational needs evolve.

8. Tiering & Slicing

- ☐ Identify customer groups with different budgets, scale, security, or performance requirements.
- ☐ Check whether the platform can **scale down**, not merely scale up.
- ☐ Support experiments and development environments where full production capabilities are unnecessary.
- ☐ Support small or gradually growing applications as well as large workloads.
- ☐ Create **vertical tiers** for different levels of operational qualities where useful.
- ☐ Allow dimensions such as performance, security, and scalability to vary independently when appropriate.
- ☐ Create **horizontal slices/modules** when customers do not need the whole platform.
- ☐ Consider offering both individual services and preconfigured integrated packages.
- ☐ Keep the underlying platform assets shared to avoid unnecessary duplication.
- ☐ Define each tier or slice around a clear use case.
- ☐ Communicate the trade-offs between tiers clearly.
- ☐ Use tiering and slicing to create a smoother entry point for customers who may grow later.

Final Review

- ☐ Are we growing **adoption**, not merely functionality?
- ☐ Are real users shaping platform evolution?
- ☐ Is the platform easy enough to start with?
- ☐ Can it continue supporting users as their needs become more complex?
- ☐ Are platform and user responsibilities clear?

- ☐ Does the roadmap protect product cohesion while remaining responsive?
- ☐ Can different customer segments consume an appropriately sized offering?
- ☐ Are our decisions and trade-offs transparent?